

Islamic Emirate of Afghanistan
Ministry of Higher Education
Directorate of Private Higher Education
RANA University
Directorate of Computer Science Faculty
Department of Software Engineering



د افغانستان اسلامي امارت
د لوړو زده کړو وزارت
د لوړو زده کړو د خصوصي پوهنتونو ریاست
رنا پوهنتون
د کمپیوټر ساینس پوهنځي ریاست
د سافت ویرانجیرنگ پیاوړتیا

MONOGRAPH PROPOSAL

ON

Darman Pharmacy Management System

سیستم مدیریت دواخانه درمان

BY

Sayed Nasrullah

22-RE200-214

In partial fulfillment of the requirements for the award of the degree of

BACHELOR OF SOFTWARE ENGINEERING

BSE

TO

RANA University

Baraki Square, Kabul–Afghanistan

Islamic Emirate of Afghanistan
Ministry of Higher Education
Directorate of Private Higher Education
RANA University
Directorate of Computer Science Faculty
Department of Software Engineering



د افغانستان اسلامي امارت
د لوړو زده کړو وزارت
د لوړو زده کړو د خصوصي پوهنتونو ریاست
رنا پوهنتون
د کمپیوټر ساینس پوهنځي ریاست
د سافت ویرانجیرنگ پیاوړتیا

MONOGRAPH PROPOSAL

ON

Darman Pharmacy Management System

In partial fulfillment of the requirements for the award of the degree of
(BSE)

TO

RANA University

TO BE PREPARED BY:

Name: Sayed Nasrullah

Father's Name: Khadem Hussain

Registration No: 22-RE200-214

Phone No: +93777413075

E-Mail: tarwanotice@gmail.com

Batch: 2022 to 2026

SUPERVISED BY:

Name:

Designation:

Qualification:

ID No:

Phone No:

E-mail:

Signature: _____

Date:

Signature: _____

Date:

Table of Contents

Introduction.....	1
Existing System... ..	1
Proposed System... ..	2
Logical Design... ..	3
Physical Design... ..	5
Tools & Technologies.....	5
Project Time & Cost Estimation.....	6
Project Timeline.....	7
PROJECT PROPOSAL APPROVAL SHEET... ..	8

1. Introduction

Community pharmacies manage medicines with different batches, costs, selling prices, suppliers, and expiry dates. When these activities are handled through notebooks, spreadsheets, or disconnected tools, the pharmacy can lose visibility over stock, expiry, purchases, and daily sales.

Darman Pharmacy Management System is proposed as a real-life, single-branch pharmacy operations system. It focuses on practical pharmacy work: medicine catalog management, batch inventory, point-of-sale sales, suppliers, purchase orders, expiry warnings, low-stock alerts, reports, settings, and role-based access.

Problem Statement and Background

Local pharmacy staff need a simple system that keeps stock accurate, prevents expired-stock mistakes, speeds up sales, preserves supplier and purchase history, and provides basic reports without requiring advanced technical knowledge.

The background of the project is the gap between daily pharmacy decisions and the quality of available information. A pharmacy owner or staff member needs to know which medicines are available, which batches should be sold first, which items are close to expiry, which items need reordering, and who performed each sensitive operation. If this information is delayed or inconsistent, the pharmacy can lose money, waste stock, or serve customers slowly.

This proposal treats the project as a real operational product, not only as a classroom exercise. The system is intentionally limited to pharmacy business operations and avoids clinical features such as patient medical records, prescriptions, insurance claims, drug-interaction checking, and telemedicine. That boundary keeps the project realistic, safer to implement, and easier to explain to pharmacy staff.

Project Aim

The aim is to design and build a secure, maintainable, and beginner-friendly pharmacy management system that can support daily pharmacy operations and demonstrate professional full-stack software engineering practice.

The expected result is a working MVP that a local pharmacy could understand and evaluate. The system should reduce manual effort, improve stock confidence, support clear staff roles, and provide enough reporting for daily operational decisions without becoming a large accounting or hospital-management system.

2. Existing System

The existing system is assumed to be manual or semi-digital. Staff may record medicines, purchases, sales, and expiry dates in paper books, spreadsheets, or separate files. This approach can work for very small operations but becomes unreliable as medicine variety and transaction volume increase.

In a manual workflow, stock information is usually updated after the counter becomes quiet or at the end of the day. This delay makes it hard to trust the current quantity during busy hours. The same issue affects purchasing because staff may reorder late, over-order slow-moving stock, or miss items that are already below threshold.

The existing approach also creates weak accountability. If a medicine record, price, supplier contact, purchase order, or stock quantity changes, there may be no clear record of who made the change and why. For a business that handles paid inventory and expiry-sensitive products, this lack of traceability is a serious operational weakness.

Area	Existing Limitation
Stock records	Manual or spreadsheet records can become outdated immediately after sales, purchases, corrections, or returned stock. Staff may know that a medicine exists but still be unsure how many saleable units are available.

Expiry control	Expiry dates are often checked only during physical inspection. This creates a risk that expired or near-expiry batches stay on the shelf while newer batches are sold first.
Sales workflow	Receipts, discounts, payment totals, and stock deduction may depend on repeated manual calculations. A busy counter can easily produce missed deductions or unclear receipt history.
Purchasing	Supplier contact records, purchase-order status, delivered quantities, and batch details are difficult to review when documents are scattered across files or notebooks.
Reporting	Daily sales, low-stock items, expiring batches, purchase value, and inventory value are slow to prepare because the data is not centralized or filtered automatically.
Security	Shared notebooks or spreadsheet files cannot enforce Admin, Pharmacist, and Cashier permissions. Staff may accidentally change records that should be protected.

Need for Change

The pharmacy needs a central system that updates stock after each protected sale or receiving operation, warns staff before stock or expiry problems become serious, and separates responsibilities between Admin, Pharmacist, and Cashier users.

The change is justified because the pharmacy does not only need digital storage; it needs controlled workflows. Sales should deduct stock automatically. Purchase receiving should create stock batches only after required delivery details are entered. Reports should be based on saved transactions rather than separate calculations. Staff permissions should prevent accidental or unauthorized changes.

3. Proposed System

The proposed system is a browser-based pharmacy management application for one branch. It supports controlled operations from login to medicine management, sales, purchasing, reporting, and settings.

The system is designed around the real responsibilities of a small pharmacy. Admin users manage the whole system, Pharmacists manage operational pharmacy data, and Cashiers focus on sales and medicine availability. This role structure reduces screen clutter and keeps sensitive changes away from users who do not need them.

The proposed solution also uses batch-normalized inventory. Stock is not stored as a single editable number on the medicine record. Instead, each received batch stores its own quantity, expiry date, cost, selling price, and supplier reference. This allows the system to support FEFO selling, expiry reports, historical receipts, and audit records.

Proposed Solution and Scope of Work

Module	Proposed Capability
Dashboard and alerts	Daily sales, transaction count, active medicines, recent sales, low-stock warnings, and expiry warnings so the pharmacy can see its operational position without preparing manual summaries.
Medicine catalog	Medicine categories, brand and generic names, dosage form, strength, SKU, barcode, unit, default prices, reorder threshold, status, and searchable medicine details.
Inventory	Batch number, received quantity, current quantity, expiry date, cost price, selling price, received date, saleable quantity, nearest expiry, and adjustment history.

Suppliers and purchases	Supplier records, contact details, active/inactive state, purchase-order drafts, ordered/cancelled states, delivery receiving, batch creation, and purchase history.
Reports and settings	Sales, inventory, expiry, and purchase reports with CSV/PDF export, plus pharmacy identity settings, receipt footer, expiry alert window, and Admin role management.

Role Scope

- Admin: full access to medicines, suppliers, purchases, sales, reports, settings, and role management.
- Pharmacist: manages medicines, inventory, sales, suppliers, and purchase orders, but cannot manage user roles or dangerous settings.
- Cashier: processes sales and checks medicine availability, without editing master data or purchasing records.

Scope Boundaries

The system is intentionally single-branch and business-focused. It does not include patient records, prescriptions, clinical decision support, insurance processing, accounting ledgers, loyalty programs, native mobile applications, AI demand forecasting, or SMS/email automation. These exclusions protect the MVP from unnecessary risk and keep the project deliverable within the academic timeline.

4. Logical Design

The logical design separates external users, application processes, data stores, and outputs. Staff requests are authenticated, authorized by role, validated in the application, and finally handled by protected database reads or transactional functions.

The main data flow begins when a staff member signs in and receives a role-based session. After that, the user can access only the modules allowed for that role. Medicine and supplier changes flow through catalog processes, purchase orders flow through purchasing processes, sales flow through POS processes, and reports read from authorized records. The system stores operational data in normalized tables and produces receipts, alerts, and exports as outputs.

The logical design also separates master data from transaction data. Medicines, categories, suppliers, settings, and profiles describe the pharmacy environment. Sales, sale items, purchase orders, purchase-order items, inventory batches, and inventory adjustments record business events. This separation makes the system easier to maintain and easier to audit.

Sales use FEFO allocation so the earliest valid batch is deducted first.

Purchase receiving creates inventory batches and adjustment logs in one protected workflow.

Reports read authorized operational data and export only the visible filtered result set.

Clinical, insurance, prescription, multi-branch, loyalty, SMS/email automation, and AI forecasting features are intentionally excluded.

4.1 Data Flow Diagram

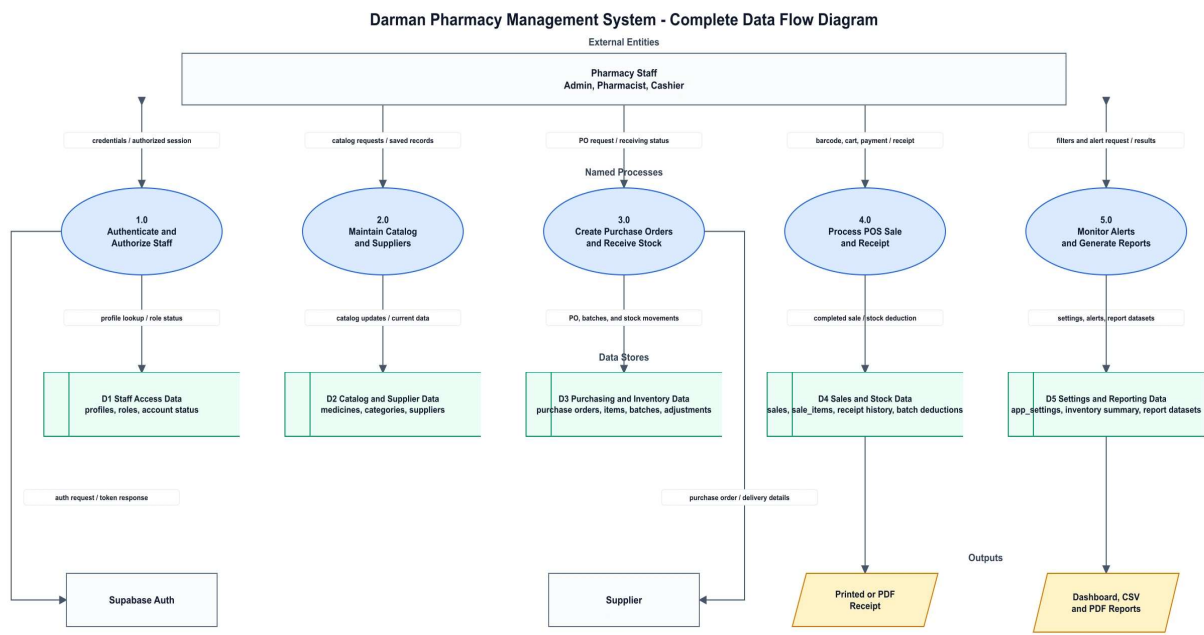


Figure 1. Complete DFD showing external entities, named processes, data stores, outputs, and labeled data flows.

4.2 Entity Relationship Diagram

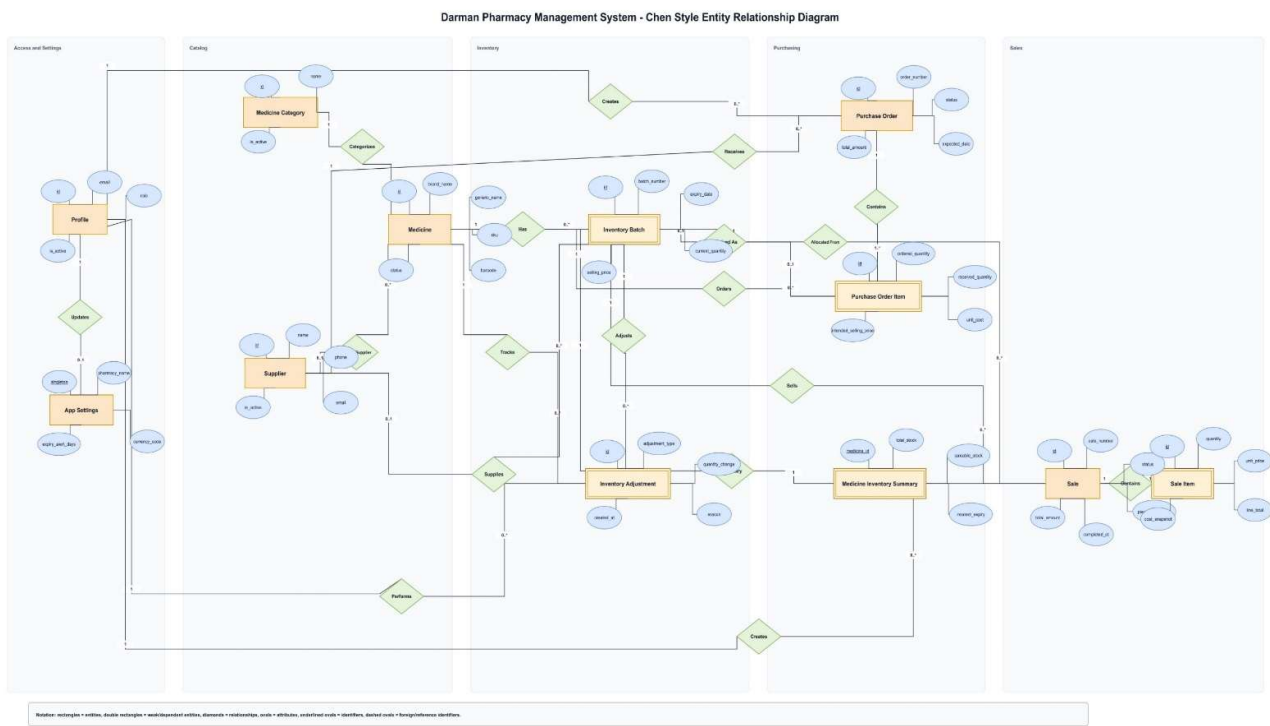


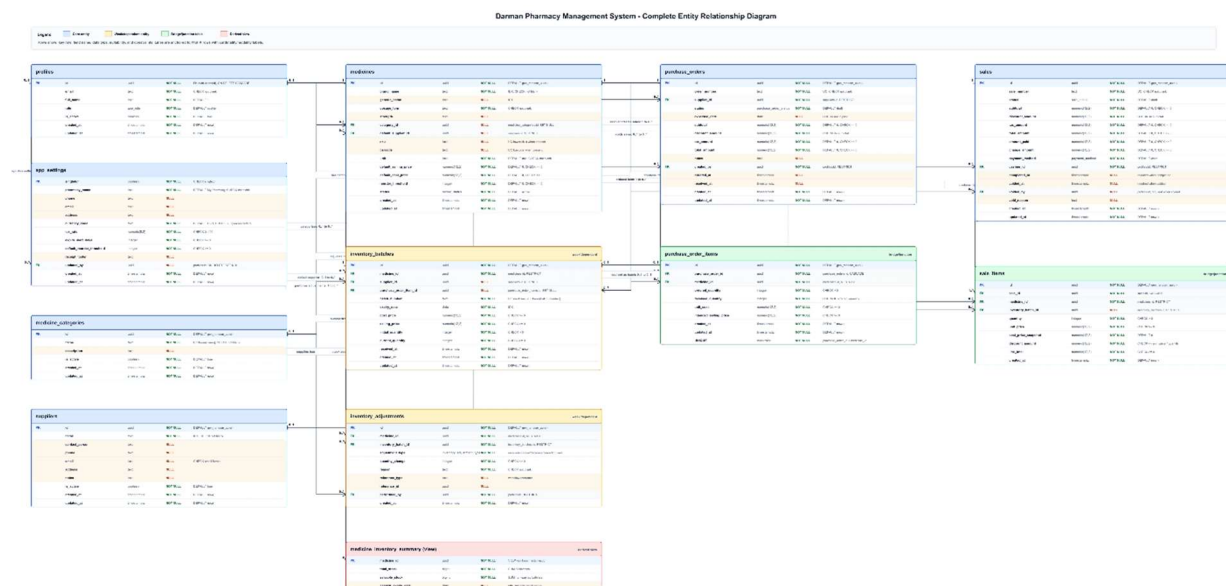
Figure 2. Complete ERD showing entities, weak/dependent tables, bridge tables, attributes, keys, data types, nullability, constraints, and cardinality.

5. Physical Design

The physical design uses a Next.js App Router frontend, typed React components, Supabase PostgreSQL, Supabase Auth, Row Level Security, views, indexes, constraints, and security-definer RPCs.

The application is organized as protected routes for dashboard, medicines, sales, suppliers, purchases, reports, settings, login, and unauthorized access. Shared UI patterns are used for page headers, tables, filters, empty states, loading states, errors, pagination, forms, dialogs, confirmations, and toast feedback. This keeps the user experience consistent across modules.

The database design uses relational constraints instead of relying only on interface rules. Prices are non-negative, quantities are positive or non-negative depending on the field, purchase discounts cannot exceed subtotals, sale completion fields are checked, batch numbers are unique per medicine, and expected purchase dates cannot be in the past. These constraints protect the data even if a client-side request is incorrect.



6. Tools and Technologies

The selected tools support a production-style academic project while staying realistic for one student developer. The stack provides type safety, reusable components, managed authentication, relational data, secure policies, report generation, and a clear deployment path.

Area	Tool or Technology	Use in the Project
Framework	Next.js 16 App Router	Routes, protected pages, layouts, production build.
Language	TypeScript	Safer application code through static typing.
Interface	React, Tailwind CSS, shadcn/ui	Responsive screens, dialogs, tables, forms, and states.
Database	Supabase PostgreSQL	Relational schema, constraints, indexes, views, and transactional RPCs.
Authentication	Supabase Auth and RLS	Email/password sessions, role-based access, and protected records.
Validation	React Hook Form, Zod	Validated forms and clear input errors.
Data and reports	TanStack Query, Recharts, jsPDF, CSV	Caching, dashboard chart, receipts, and exports.
Deployment and QA	Vercel, ESLint, TypeScript build	Hosting, linting, type checks, production build, and release validation.

7. Project Time and Cost Estimation

The estimate uses a student-budget model. Development labor is treated as academic contribution, while direct expenses cover resources needed to research, build, test, present, and demonstrate the system.

The direct budget is intentionally modest because the project uses personal development equipment and managed free or low-cost services for academic demonstration. If the system is later adopted by a real pharmacy, production hosting, backup features, domain renewal, and maintenance can be budgeted separately by the system owner.

Item	Quantity	Total USD	Basis
Student development labor	12 weeks	0	Academic contribution; not billed.
Internet and research data	3 months	90	Development, research, staging, and deployment access.
Electricity and equipment use	3 months	60	Personal computer and power usage estimate.
Custom domain	1 year	20	Optional professional project address.
Printing and binding	1 set	25	Proposal, monograph, and presentation copies.
Testing materials	1 allowance	15	Sample labels, barcode tests, and demonstration preparation.

Contingency	1 allowance	20	Unexpected academic or deployment expenses.
Total estimated direct cost		230	Student out-of-pocket project estimate.

8. Project Timeline

The roadmap organizes the project into twelve weekly milestones. Each milestone produces a concrete deliverable and supports the next project phase.

The timeline follows a practical software delivery order: first understanding the problem, then designing the data and workflows, then building the database and authentication, then implementing operational modules, and finally hardening, testing, documenting, and preparing the deployment handoff.

Week	Milestone	Main Activities	Deliverable
1	Initiation	Confirm problem, users, scope, exclusions, and success criteria.	Approved scope.
2	Existing-system study	Review manual workflows, risks, records, and expected improvements.	Problem analysis.
3	Logical design	Prepare DFD, ERD, roles, modules, and main data rules.	Validated design.
4	Database foundation	Create tables, constraints, indexes, profile automation, and RLS.	Database migration.
5	Authentication	Build login, protected routes, role landing, and access messages.	Role access.
6	Catalog and inventory	Build medicines, categories, batches, search, filters, and details.	Inventory MVP.
7	Dashboard and alerts	Build sales metrics, low-stock warnings, expiry warnings, and charts.	Dashboard.
8	Sales and POS	Build barcode/search POS, cart, FEFO checkout, receipts, and history.	Sales workflow.
9	Suppliers and purchases	Build supplier records, purchase orders, delivery receiving, and audit logs.	Purchasing workflow.
10	Reports and settings	Build exports, pharmacy settings, and role-management controls.	Admin features.
11	Hardening and QA	Run route, role, RLS, transaction, responsive, lint, type, and build checks.	Release candidate.
12	Deployment and handoff	Prepare deployment guide, client guide, final proposal, and smoke-test plan.	Final handoff.

PROJECT PROPOSAL APPROVAL SHEET

The undersigned certify that they have read the following Project Proposal and are satisfied with the overall contents and idea, and recommend the project proposal to the faculty of Computer Science for acceptance.

Title: Darman Pharmacy Management System

Prepared by: **Sayed Nasrullah**

Recommended by: **Mr. Sayed Hassan Sabit**

Vice Dean & Coordinator

Project Coordinator:

Mr. Sayed Hassan Sabit

Dean of BCS Faculty:

Mr. Abdul Ghafar Omarkhel